

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Importing the necessary libraries.

```
df = pd.read_csv('Feature Engineering Evaluation.csv')
df.info()
df.head()
df.tail()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_3731/2836160256.py in <cell line: 0>()
----> 1 df = pd.read_csv('Feature Engineering Evaluation.csv')
      2 df.info()
      3 df.head()
      4 df.tail()

----- 4 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map,
is_text, errors, storage_options)
    871     if ioargs.encoding and "b" not in ioargs.mode:
    872         # Encoding
--> 873         handle = open(
    874             handle,
    875             ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: 'Feature Engineering Evaluation.csv'
```

Next steps: [Explain error](#)

The dataset contains 5 rows and 22 columns. It contains 18 floats, 3 integers and one string.

```
df.isnull().sum()
```

The dataset was checked for empty cell. There exist no empty cell in this dataset.

```
df.columns = df.columns.str.strip()
df.drop(columns=['name', 'Unnamed: 0'])
df = df.drop(columns=['name', 'Unnamed: 0'])
df
```

The non-predictive columns (e.g., player names, IDs) was dropped so that noise or data leakage is completely removed.

```
X= df.drop(columns=['target_5yrs'])
y = df['target_5yrs']
```

target_5yrs column is clearly defined as the dependent variable while others are the independent variable.

```
corr_matrix = df.corr()
target_corr = corr_matrix['target_5yrs'].sort_values(ascending= False)
print(target_corr)
```

Correlation analysis was performed to identify highly correlated features and reduce redundancy. The three highly correlated factors are gp, min, fgm

```
df_highly_correlated = df[['target_5yrs', 'gp', 'min', 'fgm',]]
highly_correlated = ['target_5yrs', 'gp', 'min', 'fgm',]
corr_matrix = df_highly_correlated.corr()
plt.figure(figsize=(18, 8))
```

```
sns.heatmap( corr_matrix ,annot =True,cmap = 'coolwarm', xticklabels= highly_correlated ,  
            yticklabels= highly_correlated)  
plt.show()
```

```
df['ppm'] = df['pts'] / df['min']  
df['ppm']  
df
```

'ppm' is the variable to represent the column for point per minute. It calculate the division of the point column by the minutes column.

Start coding or [generate](#) with AI.